
How to Pick the Model: A Framework for Budget-Aware Orchestration

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Agent systems on verified tasks invite a deceptively simple question: which back-
2 bone model should be run? The strongest model is not always the best deployment
3 choice, because verified progress must be bought under finite wall-clock, token,
4 API, and evaluation budgets. We formulate this problem as deployment-aware
5 model selection. A fixed router observes a pre-deployment information state and
6 chooses a worker or harness action; the decision is scored by a checker-indexed
7 deployment loss rather than by routing accuracy or final checker score alone. The
8 key estimand is paired: on the same task instances, with the same router and action
9 menu, does richer information change the selected action and reduce net deploy-
10 ment loss after its own measurement and context costs are charged? We instantiate
11 the framework on an Karpathy’s AutoResearch-inspired CIFAR-10 edit-verify
12 benchmark, where a prompt-only router chooses among heterogeneous workers
13 before costly deployment runs. The protocol reports when checker-score rank-
14 ings disagree with deployment-loss rankings, when pre-deployment information
15 changes routing, and whether those changes pay for themselves. The resulting
16 protocol evaluates not which agent is best in isolation, but when information is
17 worth buying before deploying one.

18 1 Introduction

19 Practitioners building agent systems face multiple practical challenges: what model (backbone) to
20 use, how to route between models, when to spawn sub-agents, how to engineer context between them,
21 and so on. All these possible choices often affect performance (quality of the solution), speed (time to
22 solution), and the mere token cost. The question is how to pick between the different possible designs.
23 The common way to answer this question is benchmark racing: take a benchmark as SWE-bench
24 [1] or MLE-bench [?] and pick the agent system with best average end-to-end performance. An
25 increasingly long line of work, based on this "recipe", is proposing better and better agentic system
26 architecture to achieve SOTA at these tasks. This article investigates:

27 *whether there exist other ways to answer the question of what the most suited agent system is.*

28 Indeed, (i) a large number of tasks for which such benchmark does not exist (as developing academic
29 research [?]). And (ii) very often, the task presents instances of varied difficulty and the practitioner
30 may want to understand whether on the single instance a lower-cost, faster model is sufficient; more
31 tools should be given to the systems; or a more involved orchestration is required to solve the instance.
32 As a running example for the rest of the paper, the question whether the user should use on their own
33 task Haiku vs Sonnet vs Opus in Claude Code, or Claude Code vs Codex, is not fully addressed by
34 benchmark racing.

The central issue is that one benchmark score may conflate mechanisms that imply different engineering decisions. A system can win because its worker takes lower per-step time or token-count. Because the selected worker is more competent on a solver-relevant task mode. Analogously, imagine some sort of pre-deployment signal (problem definition, the context, or an initial test) reveals information about the instance which can be used to better orchestrate or to choose what backbone to pick. A system can win if it *can* orchestrate or choose the model optimally exploiting that information or not—as humans implicitly do when they decide to use lower or higher reasoning or different models on their tasks.

More practically, in this article *we study the deployment problem of choosing which model or harness to run on a verifiable task*. The objective is time to a required verified result: among actions that may eventually succeed, the useful one is the one expected to reach the verifier threshold fastest, after charging any pre-deployment measurement used to make the choice. Achille and Soatto [2], indeed, recently showed that for verifiable tasks the time to solution, as in their *Proper Time* [2, Definition ?] is the performance measure to compare them. Building on this intuition we operationalize the difference in time-to-solution of two possible designs. We show it *mathematically* decomposes in 4 components which isolates the mechanisms of choice mentioned above. The experimental protocol then shows these can be estimated and lead to improvements in speed on Karpathy’s AutoResearch tasks.

Research questions and contributions. We organize the paper around three questions, each paired with the contribution that answers it.

1. **Can the time-to-solution gap between agent designs be decomposed into meaningful terms?** Theorem 1 gives an exact four-term identity under a packed latent-mode assumption: cost, within-mode competence, routing information, and allocation-summary mismatch additively explain the improvement in expected certified log-effort.
2. **Can the decomposition replace end-to-end racing for model selection?** Algorithm 1 and the pilot concentration bound in Appendix A turn the identity into a structured pilot protocol whose evaluation cost depends on the number of models and task modes.
3. **When does a lower-cost model with retries beat a stronger model?** Theorem 2 gives the single-mode crossover rule, and the multi-mode accounting shows why the answer changes across task modes.

The experiments below instantiate these claims with checked trajectories and deployment accounting.

2 Mathematical Framework for Agent Designs on Verifiable Tasks

We study the problem faced by a deployment system before launching an agentic run: given a task instance and a finite menu of possible workers or harnesses, which one should be run? The answer is not necessarily the model with the best final score. A deployment action must produce checked progress under finite time, token, API, and evaluation budgets.

Definition 1 (Checked deployment task). *A checked deployment task is a tuple*

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, \mu),$$

where $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is an artifact or solution space, V is an external checker or scoring procedure on instance–artifact pairs, and μ is the deployment distribution over instances. For a realized instance $x \in \mathcal{X}$, an agent produces artifacts $y \in \mathcal{Y}$ that are scored by $V(x, y)$.

The checker may be binary, as in unit tests or proof checking, or scalar, as in validation loss or reward. The key point is externality: progress is determined by the checker, not by the model’s self-report.

Definition 2 (Actions and routers). *Let $\mathcal{A}$ be a finite menu of deployable actions. An action may be a single model, a fixed model harness, a retry policy, or a tool-using agent protocol. For action $a \in \mathcal{A}$ and instance x , let*

$$\ell_{\text{dep}}(a, x; \xi)$$

be the predeclared deployment loss of running action a on x , where ξ denotes deployment randomness such as sampling, system latency, or data-order randomness. The loss may include checked progress, failure, persistence above threshold, wall-clock time, token/API cost, and checker/evaluation cost.

83 A router R observes a permitted pre-deployment information state $W(x)$ and selects an action

$$a_R(x) = R(W(x)) \in \mathcal{A}.$$

84 The selected action then performs the deployment run from the original task state.

85 This separates routing from deployment. The router chooses what to run; the selected action performs
86 the actual checked run. Unless explicitly declared otherwise, pre-deployment records are shown only
87 to the router and are not passed to the selected worker.

88 3 Verifiable agent design

89 In this section we set the notations and introduce the mathematical framework to be able to deal with
90 agent systems on verifiable tasks.

91 **Definition 3** (Budgeted externally checked task). A budgeted externally checked task is a tuple

$$\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\text{dep}}, \mu).$$

92 Here $\mathcal{X}$ is an instance space, $\mathcal{Y}$ is a solution space, $V : \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{R}$ is an external verifier or
93 scoring procedure, B_{dep} is the full deployment budget and acceptance object, and μ is the deployment
94 distribution over instances. The object B_{dep} may include a horizon, verifier budget, token/time/cost
95 caps, and a thresholding rule that induces a binary success event from $V(x, y)$. An agent succeeds
96 on $x \sim \mu$ if it produces an accepted y before exhausting B_{dep} .

97 **Definition 4** (Workers, actions, and routed designs). Let $\mathcal{M} = \{M_1, \dots, M_K\}$ be a finite menu
98 of workers. A worker is any deployable solver or fixed agent policy that can produce a candidate
99 solution for verification. Let $\mathcal{A}$ be a finite menu of deployable actions. In the simplest setting, $\mathcal{A} = \mathcal{M}$;
100 more generally, an action bundles a worker with fixed stopping, retry, tool-use, and carryover rules.
101 A routed design is a tuple

$$d = (\mathcal{A}, R, \kappa),$$

102 where R is a router that selects an action after observing permitted information and κ is the declared
103 scalar action-cost convention used in the log-effort diagnostic. When costs are logged as resource
104 vectors, the protocol must fix the scalar reduction before evaluation. A signal condition induces the
105 routing policy

$$\rho_j : x \mapsto a_j(x) = R(Z_j(x)) \in \mathcal{A}.$$

106 The router is part of the deployed system, but it is evaluated through the action it selects. Pre-
107 deployment records are passed to the selected action only if this is declared as part of that action.

108 **Definition 5** (Task instances and task modes). Given $\mathcal{T} = (\mathcal{X}, \mathcal{Y}, V, B_{\text{dep}}, \mu)$, a task instance is
109 a concrete realization $X = x \in \mathcal{X}$ of that same task. It is the measurable object supplied to the
110 deployed system; the router and worker may access only the fields made available by the protocol. A
111 task mode is a latent variable

$$S = \sigma(X) \in \mathcal{S} = \{1, \dots, m_S\}$$

112 induced by a measurable map $\sigma : \mathcal{X} \rightarrow \mathcal{S}$ that assigns each instance to one of finitely many solver-
113 relevant variants of the task. It defines a prior $\pi_s = \mathbb{P}_\mu[S = s]$ and mode-conditional instance
114 distributions $\mu_s = \mu(\cdot \mid S = s)$. Modes are therefore not different tasks and not worker labels. They
115 are variants of the same task: finite subpopulations of instances that share solver-relevant structure
116 and may favor different workers or interventions.

117 The mode variable is retained by the evaluator for accounting and diagnostic comparisons. It need not
118 be observed by either the router or the worker. This lets the same theory cover expert-defined task-
119 mode schemas and finite evaluator-defined modes. When an experiment uses one fixed executable
120 prototype per mode, it estimates conditional panel quantities rather than the population quantities
121 induced by μ .

122 **Assumption 1** (Packed latent-mode family). Each instance $X \sim \mu$ is associated with a mode
123 $S = \sigma(X)$ in a finite mode set $\mathcal{S}$, with prior π . Conditional on mode s , action a has success
124 probability $p(a, s) > 0$ and scalar cost

$$\kappa(a, s) = \mathbb{E}[C_\delta(a, X; \xi) \mid S = s] > 0$$

under the declared deployment protocol, where C_δ is the normalized cost accumulated to the first certified success or the horizon. At deployment time, the router observes a signal $Z = z$ and selects one action. The evaluator defines the mode distribution induced by that signal,

$$\pi_z(s) = \mathbb{P}[S = s \mid Z = z].$$

The router is not required to output π_z or any mode distribution. Instead, the theory uses a fixed evaluator-side allocation summary $q_z \in \Delta(\mathcal{S})$, with $q_z(s) > 0$ whenever $\pi_z(s) > 0$. This summary is not a router output; it is a theory-facing diagnostic distribution specified before evaluation. Write $a_z = \rho(z)$ for the action selected after observing z . For an instance whose true mode is s , the certified effort surrogate is proportional to

$$T_{\rho,q}(s, z) \propto \frac{\kappa(a_z, s)}{p(a_z, s) q_z(s)}. \quad (1)$$

Assumption 1 is deliberately restrictive. Real task instances can vary in infinitely many ways, while the theory uses a finite state abstraction. The assumption says that, for model selection, instances can be packed into modes with stable action success probabilities. The purpose is to isolate the regime in which cost, competence, information, and allocation-summary mismatch are algebraically separable and empirically testable. Residual and calibration diagnostics measure when this finite packing is too coarse.

Definition 6 (Cost-adjusted certified log-effort diagnostic objective). For a fixed routed policy $\rho : Z \mapsto a_Z$ and a fixed evaluator summary q satisfying Assumption 1, define

$$\mathcal{J}(\rho, q) = \mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} [\log \kappa(a_Z, S) - \log p(a_Z, S)] + \mathbb{E}_Z [\mathcal{H}(\pi_Z) + \text{KL}(\pi_Z \| q_Z)]. \quad (2)$$

Lower $\mathcal{J}$ means lower expected certified log-effort under the fixed diagnostic convention. The quantity q is not optimized by the router; it is fixed by the evaluator-side diagnostic rule. When several signal conditions are compared, $\mathcal{J}$ is applied to the corresponding policy summaries; Appendix C writes this comparison explicitly for Z_j versus Z_0 .

The action quantities in (2) are $\kappa(a_Z, s)$ and $p(a_Z, s)$: how costly the selected action is on mode s and how often it succeeds there. The router and information quantities are Z , π_z , and q_z : what the router is allowed to observe, what that signal reveals about the task mode, and the theory-facing allocation summary used in the KL term.

Remark 1 (Meaning of the effort surrogate). Equation (1) says that, on a true mode s after signal z , certified effort increases with action cost, decreases with action success probability on that mode, and increases when the allocation summary gives too little diagnostic mass to that mode. The router is not literally routing jobs to modes; modes are evaluator-defined task variants. The quantity $q_z(s)$ is a diagnostic summary, not a deployment probability. The KL term below is therefore an allocation-summary mismatch under the chosen diagnostic convention. It should be interpreted as router mismatch only if the experiment predefines a reproducible construction of q_z from router allocation behavior.

4 Performance accounting

For a fixed signal value z , the routing part of the objective is the cross-entropy between the signal-induced mode distribution and the fixed allocation summary:

$$\text{CE}(\pi_z, q_z) = - \sum_{s \in \mathcal{S}} \pi_z(s) \log q_z(s) = \mathcal{H}(\pi_z) + \text{KL}(\pi_z \| q_z). \quad (3)$$

The KL term is a divergence over finite task-mode distributions. It is not a token-level language-model divergence.

Theorem 1 (Four-term routed-policy decomposition). Under Assumption 1, compare a baseline policy ρ_0 that deploys fixed action a_0 without using Z and uses $q_0(\cdot \mid z) \equiv \pi$, with a deployed routed policy summarized by (ρ, q) . The improvement in expected certified log-effort, $\Delta = \mathcal{J}(\rho_0, q_0) - \mathcal{J}(\rho, q)$, decomposes as

$$\Delta = \underbrace{\mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} \left[\log \frac{\kappa(a_0, S)}{\kappa(a_Z, S)} \right]}_{\text{cost}} + \underbrace{\mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} \left[\log \frac{p(a_Z, S)}{p(a_0, S)} \right]}_{\text{competence}} + \underbrace{\mathbb{I}_\pi(S; Z)}_{\text{routing information}} - \underbrace{\mathbb{E}_Z [\text{KL}(\pi_Z \| q_Z)]}_{\text{allocation-summary mismatch}}. \quad (4)$$

166 *Proof sketch.* Taking logs in (1) gives $\log T_{\rho,q}(s, z) = C + \log \kappa(a_z, s) - \log p(a_z, s) - \log q_z(s)$.
 167 Conditioning on $Z = z$ and averaging over $S \sim \pi_z$ converts $-\mathbb{E} \log q_z(S)$ into the cross-entropy
 168 in (3). Subtracting the prior-matched baseline cancels constants and uses $H(\pi) - \mathbb{E}_Z H(\pi_Z) =$
 169 $I_\pi(S; Z)$. The full algebra is given in Appendix A. $\square$

170 The four terms have different operational meanings. If the cost term is large and positive, a lower-cost
 171 action may be preferable even when it reaches success later in step count. If the competence term
 172 varies by mode, a global worker ranking is likely to hide comparative advantage. If information gain is
 173 high, the pre-deployment signal exposes solver-relevant structure. If allocation-summary mismatch is
 174 high, the signal-induced mode distribution is poorly matched to the evaluator’s predeclared allocation
 175 summary. For two signal conditions, the identity compares the routed policies induced by those
 176 signals. A richer signal is useful only if its induced action change improves the cost–success tradeoff
 177 after the cost of obtaining the signal is charged.

178 **Operational reading.** The task mode S is not an oracle label for the best worker. The evaluator
 179 fixes a mode schema before evaluation, then estimates the mode-conditional quantities $p(a, s)$ and
 180 $\kappa(a, s)$. Routing information asks whether a signal makes those modes more identifiable. Routing
 181 shift asks whether the selected action changes when the signal is enriched. Allocation-summary
 182 mismatch is the KL term defined by the fixed allocation summary q . Allocation regret, used later
 183 in the empirical protocol, is a separate action-frontier diagnostic rather than an estimator of this KL
 184 term. The final decision is not made by the information term alone; it is made by paired deployment
 185 gain on held-out instances after measurement and context costs are charged. Appendix B gives the
 186 full operationalization of these diagnostics.

187 4.1 When lower-cost retries win

188 **Theorem 2** (Single-mode retry crossover). *Fix a single homogeneous mode, success threshold δ ,*
 189 *and horizon H . Let actions h and ℓ have one-attempt first-passage success probabilities $p_h =$
 190 $\mathbb{P}[\tau_\delta(h, X) \leq H]$ and $p_\ell = \mathbb{P}[\tau_\delta(\ell, X) \leq H]$, with expected first-passage costs κ_h and κ_ℓ . Assume
 191 $0 < p_\ell < p_h < 1$ and $\kappa_\ell < \kappa_h$. Assume retries of ℓ are conditionally i.i.d. fresh attempts, each with
 192 the same one-attempt horizon H and common Bernoulli success probability p_ℓ . The lower-cost retry
 193 depth that matches the strong action’s one-attempt success probability is*

$$D_{\text{cross}} = \frac{\log(1 - p_h)}{\log(1 - p_\ell)}.$$

194 The corresponding integer fixed-depth retry block is cost-favorable whenever

$$\lceil D_{\text{cross}} \rceil \kappa_\ell < \kappa_h.$$

195 **Remark 2** (Stopping after first retry success). *If retries of the lower-cost action stop after the first*
 196 *success and each attempted retry has unconditional expected cost κ_ℓ , the expected cost at depth D is*

$$\kappa_\ell \sum_{d=1}^D (1 - p_\ell)^{d-1} = \kappa_\ell \frac{1 - (1 - p_\ell)^D}{p_\ell},$$

197 and the decision should also account for the residual failure probability under the deployment loss.
 198 Any difference in within-attempt time-to-success is already absorbed into κ_h and κ_ℓ .

199 The multi-mode case is where the four-term identity matters. A lower-cost worker can be rational
 200 on one task mode and irrational on another. A router is useful only when different actions win the
 201 cost–success tradeoff on different modes and the pre-deployment information helps identify which
 202 mode applies.

203 5 Decomposition-guided selection

204 The decomposition suggests a practical protocol. Rather than evaluate every complete design as an
 205 independent arm, the builder first estimates shared quantities: mode-conditional success, cost, signal
 206 information, and allocation diagnostics. Then a larger finite design menu can be scored analytically,
 207 with expensive deployment evaluations reserved for finalists and for paired validation.

Figure placeholder. Final figure: left-to-right accounting pipeline. Inputs are checked task instances, finite task modes, worker or harness actions, admissible pre-deployment information, and cost logs. The center computes the four log-effort terms. The right panel reports deployment-loss validation: rank flips, paired router gain, negative controls, and calibration diagnostics.

Figure 1: The V3 framing keeps the four-term identity as the theory-facing diagnostic and tests it with deployment-loss outcomes.

208 The log-effort objective $\mathcal{J}$ is a diagnostic surrogate, not the deployment loss. For deployment
209 validation we use a predeclared first-passage loss. Given a checked progress trace $G_1, \dots, G_H$,
210 define

$$\tau_\delta(a, x; \xi) = \inf\{t \leq H : G_t(a, x; \xi) \geq \delta\}, \quad F_\delta(a, x; \xi) = \mathbf{1}\{\tau_\delta(a, x; \xi) = \infty\},$$

211 and

$$\text{Occ}_{\delta, H}(a, x; \xi) = H^{-1} \sum_{t=1}^H \mathbf{1}\{G_t(a, x; \xi) \geq \delta\}.$$

212 Let $c_\delta(a, x; \xi)$ be the normalized worker-side resource vector accumulated to $\tau_\delta \wedge H$, and let

$$C_\delta(a, x; \xi) = \lambda^\top c_\delta(a, x; \xi)$$

213 be the scalar cost used in $\kappa(a, s)$. The primary loss is

$$\ell_{\text{dep}}(a, x; \xi) = \alpha_{\text{fail}} F_\delta(a, x; \xi) + C_\delta(a, x; \xi), \quad (5)$$

214 with $\alpha_{\text{fail}} = 1$ and normalized resource weights λ fixed before evaluation. This is the loss used
215 in $\mathcal{J}_R(j)$ and $\Delta_R(j)$ unless a different primary loss is explicitly declared. Because the benchmark
216 records full trajectories to horizon H , we also report the full-horizon audit loss

$$\ell_{\text{audit}}(a, x; \xi) = \ell_{\text{dep}}(a, x; \xi) + \alpha_{\text{occ}}(1 - \text{Occ}_{\delta, H}(a, x; \xi)) + \alpha_{\text{qual}}\psi(G_H(a, x; \xi)), \quad (6)$$

217 where $\alpha_{\text{occ}} = \alpha_{\text{qual}} = 1$ and $\psi(u) = 1 - u$ after clipping $G_H \in [0, 1]$. The audit loss checks
218 persistence and final quality; it does not change the first-passage cost convention in Theorem 1.

219 **Algorithm 1** (Theory-to-deployment selection protocol).

Input: Task family, checker, finite task-mode schema $\mathcal{S}$, action menu $\mathcal{A}$, admissible progressive signals $\{Z_j\}$, pilot cell count n_{cell} , deployment repeats Q_{dep} , confirmation budget.

Output: Recommended signal-conditioned deployment policy, reported accounting terms, paired deployment gain, and calibration diagnostics.

- 1 *Freeze the modes and loss.* Declare $\mathcal{S}$, success threshold δ , horizon H , cost normalizers, deployment loss (5), measurement costs, and router output contract.
- 2 *Structured pilot.* For each action $a \in \mathcal{A}$ and mode $s \in \mathcal{S}$, estimate $\hat{p}(a, s)$, $\hat{\kappa}(a, s)$, first-hit curves, occupancy, and uncertainty intervals.
- 220 3 *Signal and router diagnostics.* For each progressive signal Z_j , report any predeclared signal-information diagnostic and log the action allocation induced by the router.
- 4 *Accounting score.* Compute the four log-effort terms from Theorem 1 only for quantities whose diagnostic estimators were predeclared; otherwise report allocation regret separately.
- 5 *Paired deployment validation.* On the same holdout instances, run the selected action under Z_0 and richer Z_j conditions, score repeated selected-worker trajectories by (5), and subtract measurement cost.
- 6 *Decision.* Choose the signal condition with the largest held-out $\hat{\Delta}_R(j)$ subject to predeclared gates; use $\mathcal{J}$, frontier scores, and allocation regret as diagnostics, not as the final deployment objective.

221 5.1 Progressive signals and paired router value

222 Let $Z_j(X; \zeta_j)$ be the router-visible pre-deployment record under signal condition j , where ζ_j denotes
223 measurement randomness. Let Z_0 be the static signal observed before choosing a deployment action.

For any admissible richer condition j , the router observes Z_j , pays the corresponding measurement and router-context cost, and selects an action

$$a_R^j(X) = R(Z_j(X; \zeta_j); U_R) \in \mathcal{A}.$$

Here U_R denotes router-side randomness; for deterministic routers it is fixed and suppressed. Thus the experimental comparison is between routing policies ρ_0, ρ_j induced by different signals for the same orchestrator R , not between different orchestrators. The normalized measurement loss is

$$\ell_{\text{meas}}(j, x; \zeta_j) = \lambda_{\text{meas}}^\top d_j(x; \zeta_j),$$

where d_j contains measurement wall-clock, tokens, checker calls, context expansion, parsing, retry, and latency costs. Set $\ell_{\text{meas}}(0, x) = 0$. The weights λ_{meas} use the same predeclared normalizers as deployment resource cost unless a separate router-cost scale is declared. In the main protocol, pre-deployment probes are outside the selected action's deployment run: they are charged through ℓ_{meas} and do not reduce the later horizon H or verifier budget B_V .

The selected-action deployment objective is

$$\mathcal{J}_R(j) := \mathbb{E} \left[\ell_{\text{meas}}(j, X; \zeta_j) + \ell_{\text{dep}}(a_R^j(X), X; \xi) \right]. \quad (7)$$

The expectation averages over task instances, measurement randomness, router randomness, and deployment randomness unless a conditional estimand is explicitly declared. The router-facing value of signal condition j is the paired gain

$$\Delta_R(j) := \mathcal{J}_R(0) - \mathcal{J}_R(j) = \mathbb{E} \left[\ell_{\text{dep}}(a_R^0(X), X) - \ell_{\text{dep}}(a_R^j(X), X) - \ell_{\text{meas}}(j, X) \right]. \quad (8)$$

Positive $\Delta_R(j)$ means that the richer signal improves net deployment loss for the same router on the same task distribution. Allocation shift alone is insufficient:

$$\text{Shift}_R(j) = \Pr[a_R^j(X) \neq a_R^0(X)]$$

measures decision relevance, not benefit.

Proposition 1 (Unbiased paired router-gain estimator). *Assume task instances are sampled i.i.d. from μ , the orchestrator, action menu, and signal protocols are fixed before evaluation, measurement costs are recorded without bias, and deployment seeds are independent conditional on selected action and instance. The router is run once per instance and signal condition; the Q_{dep} deployment repeats estimate the conditional expected loss of the selected action. For condition j , estimate*

$$\hat{\Delta}_R(j) = \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{Q_{\text{dep}}} \sum_{q=1}^{Q_{\text{dep}}} \ell_{\text{dep}}(a_{R,i}^0, x_i; \xi_{i,q}^0) - \frac{1}{Q_{\text{dep}}} \sum_{q=1}^{Q_{\text{dep}}} \ell_{\text{dep}}(a_{R,i}^j, x_i; \xi_{i,q}^j) - \hat{\ell}_{\text{meas}}(j, x_i) \right].$$

If the repeated-run averages are unbiased for the expected deployment losses of the selected actions, then $\mathbb{E}[\hat{\Delta}_R(j)] = \Delta_R(j)$.

When the same worker is selected under Z_0 and Z_j , using common deployment seeds preserves unbiasedness and reduces paired variance. When counterfactual outcomes for all actions are logged on the same instance, hindsight action regret can be reported as a diagnostic, but it is not the primary estimand. If the study fixes a finite prototype panel rather than sampling i.i.d. instances from μ , the same estimator describes that panel and should not be interpreted as a population-level deployment estimand.

6 Operational experimental protocol

This section specifies the controlled protocol used to test the routed-policy decomposition. The main study is a fixed-prototype repeated-run panel: it fixes three AutoResearch task modes and uses repeated agentic trajectories to estimate, for the same starting executable artifact, how worker choice, signal exposure, and routing affect first-passage success and cost. Accordingly, the primary estimates are panel analogues of the population quantities in Section 4. Aggregate panel summaries use the predeclared empirical mode weighting $\hat{\pi}(s) = 1/3$ for $s \in \mathcal{S}$.

Task modes. The task is AutoResearch-style executable-artifact optimization on CIFAR-10. Each task instance contains an editable `train.py`, a read-only data/checker module `prepare.py`, and a verifier that executes `train.py` and reports task-level diagnostics for validation performance, compute cost, optimization progress, and model complexity. In this protocol, each task mode is represented by one fixed executable prototype of the same CIFAR-10 task. Concretely, a task mode is specified by the tuple

$$\chi_s = (A_s, D, \text{split}, u, \eta_s, V, B_V, H, \delta),$$

where A_s is the starting model class and editable `train.py` template, D is CIFAR-10, `split` is the fixed train/validation split, u is the model-initialization seed, η_s denotes starting optimization and architecture hyperparameters, V is the checker, B_V is the checker budget, H is the agent horizon, and δ is the success threshold. The protocol uses three task modes:

$$S = \{\text{mlp_flat}, \text{cnn_compact}, \text{resnet_micro}\}.$$

Execution protocol. Appendix D reports the full fixed configuration read from the current `train.py` templates and the CIFAR-10 checker in Table 3. The paper-facing worker menu is

$$\mathcal{M}_{\text{main}} = \{\text{gpt_5_4_mini}, \text{gpt_5_3_codex}, \text{gpt_5_4}\}.$$

In the main experiment, each deployable action is one worker under the common fixed prompt and harness, so $\mathcal{A} = \mathcal{M}$. In the current snapshot, `gpt\_5\_3\_codex` and `gpt\_5\_4` are the mature comparison pair. `gpt\_5\_4\_mini` is being filled in as the lower-cost replacement for the unavailable `gpt\_5\_3\_codex\_spark` condition. Each worker uses the same prompt. For every task-mode-worker cell, we run

$$n_{\text{cell}} = 10$$

independent trajectories from the same fixed `train.py`; the repeated-run index changes only the agentic trajectory. The router is a fixed prompt-only orchestrator that chooses one worker from $\mathcal{M}$ before deployment. For the primary protocol it is deterministic after fixing model identifier, prompt, parser, temperature, and seed, so $B_R = 1$ router draw is logged per task-mode-signal pair. We evaluate a sequential signal protocol

$$Z_0, Z_1, Z_2, Z_3.$$

The signals are nested: Z_j contains all fields of Z_{j-1} plus the additional fields declared for condition j . Z_0 contains only the task name, checker objective, worker menu, and budget. Z_1 adds the initial `train.py` and a static workload summary: starting model class, parameter count, optimizer and budget settings, dataset/checker identifiers, and parse/runtime availability. Z_2 adds a low-cost checker probe of the unmodified artifact at a predeclared probe budget. Z_3 adds the full unmodified-artifact baseline under the deployment checker budget. Probe records contain validation loss, validation accuracy, training seconds, total seconds, optimizer steps, parameter count, parse/runtime status, and the probe budget. Probe runs are measurement actions for the router only: they are charged through ℓ_{meas} and do not consume the selected worker’s later deployment horizon or checker budget. Let $L_{\text{base}}(x)$ be the validation loss of the unmodified starting artifact for instance x under the fixed deployment checker. This evaluator-only baseline is used to score G_t in all conditions; it becomes router-visible only in Z_3 and is then charged as measurement information. In the fixed-prototype protocol this baseline is constant within each task mode. At edit step t , a worker produces a candidate `train.py`, the checker returns loss L_t , and checked progress is

$$G_t = \text{clip}_{[0,1]} \left(\frac{L_{\text{base}}(x) - L_t}{|L_{\text{base}}(x)| + \varepsilon_L} \right).$$

We set $\varepsilon_L = 10^{-8}$ in the primary analysis. The primary success event is first passage above the relative-improvement threshold

$$\tau_\delta = \inf\{t \leq H : G_t \geq \delta\}, \quad \delta = 0.05.$$

The fixed-prototype competence estimator is $\hat{p}_{\chi_s}(a) = n_{\text{cell}}^{-1} \sum_{r=1}^{n_{\text{cell}}} \mathbf{1}\{\tau_{\delta,a,s,r} \leq H\}$, the panel analogue of $p(a, s)$. The primary cost $\hat{\kappa}_{\chi_s}(a)$ is the sample mean worker wall-clock accumulated to $\tau_\delta \wedge H$, the panel analogue of $\kappa(a, s)$; token count is reported as a sensitivity analysis. The primary deployment loss is the first-passage loss in (5); threshold occupancy and final checked quality are reported through the audit loss (6). For log scores, we use the predeclared smoothed success estimate $\tilde{p}_{\chi_s}(a)$ to avoid infinite values in zero-success pilot cells.

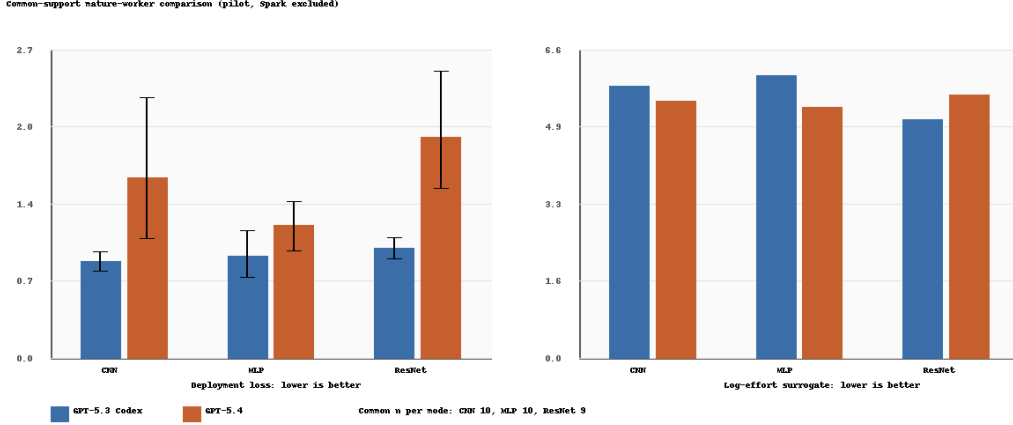


Figure 2: Mature-worker snapshot on common pilot support, excluding the rate-limited Spark condition. Left: deployment loss, where lower is better and intervals are the bootstrap intervals produced by the accounting script. Right: the certified log-effort surrogate used by the decomposition. The key qualitative result is a rank disagreement: gpt_5_4 reaches the threshold faster and achieves higher final improvement, but the deployment-loss accounting currently favors gpt_5_3_codex in all three modes, and the log-effort surrogate favors different workers by mode.

Table 1: Pilot worker-frontier summary on common support. n_{common} is the number of paired trial indices retained for the two mature workers in that mode. The table reports first-hit success at $\delta = 0.05$, mean first-hit step among successful runs, mean threshold occupancy, mean final relative improvement, the log-effort surrogate, and deployment loss. Lower is better for the last two columns.

Mode	Worker	n_{common}	Success	Mean τ	Occupancy	Final improv.	Log-effort	Dep. loss
cnn_compact	gpt_5_3_codex	10	10/10	6.50	0.725	0.232	5.814	0.848
cnn_compact	gpt_5_4	10	10/10	3.30	0.885	0.373	5.503	1.580
mlp_flat	gpt_5_3_codex	10	9/10	8.11	0.580	0.171	6.045	0.890
mlp_flat	gpt_5_4	10	10/10	2.60	0.920	0.267	5.358	1.158
resnet_micro	gpt_5_3_codex	9	10/10	3.20	0.890	0.194	5.102	0.962
resnet_micro	gpt_5_4	9	9/9	1.67	0.967	0.282	5.626	1.934

305 **Experiment 1: worker frontier.** Run all workers on all three task modes without a router:

$$3 \text{ task modes} \times 3 \text{ workers} \times n_{\text{cell}} = 90$$

306 full trajectories. Estimate $\hat{p}_{\chi_s}(a)$, $\hat{\kappa}_{\chi_s}(a)$, mean first-hit step, occupancy, final quality, and uncertainty
307 intervals. For each task mode, construct the empirical cost–success frontier

$$\hat{\mathcal{F}}(s) = \left\{ a \in \mathcal{A} : \log \hat{\kappa}_{\chi_s}(a) - \log \hat{p}_{\chi_s}(a) \leq \min_{a' \in \mathcal{A}} [\log \hat{\kappa}_{\chi_s}(a') - \log \hat{p}_{\chi_s}(a')] + \epsilon_{\text{frontier}} \right\},$$

308 where $\epsilon_{\text{frontier}}$ is a predeclared uncertainty tolerance. This experiment is the worker-side calibration
309 for Theorem 1. It estimates fixed-prototype analogues of $\kappa(a, s)$ and $p(a, s)$ for every candidate action
310 before any router is allowed to choose. The same estimates can be plugged into Theorem 2, because
311 the first-passage success probabilities and first-passage costs determine when repeated deployments
312 of a lower-cost worker can match the one-attempt success probability of a stronger worker.

313 **Current worker-frontier results.** Because the Spark condition hit rate limits, the main snapshot
314 below compares the two mature workers only: gpt_5_3_codex and gpt_5_4. For each mode, we
315 use the maximum common support available to both workers: ten pilot trajectories for cnn_compact,
316 ten for mlp_flat, and nine for resnet_micro. The provisional gpt_5_4_mini pilot is intentionally
317 not pooled into these estimates; it is reserved for the final lower-cost comparison once the cell reaches
318 the same support.

319 This is the first empirical support for the paper’s main claim. If one ranks workers by raw final
320 improvement or by first-hit speed, gpt_5_4 looks dominant. If one ranks by the deployment loss

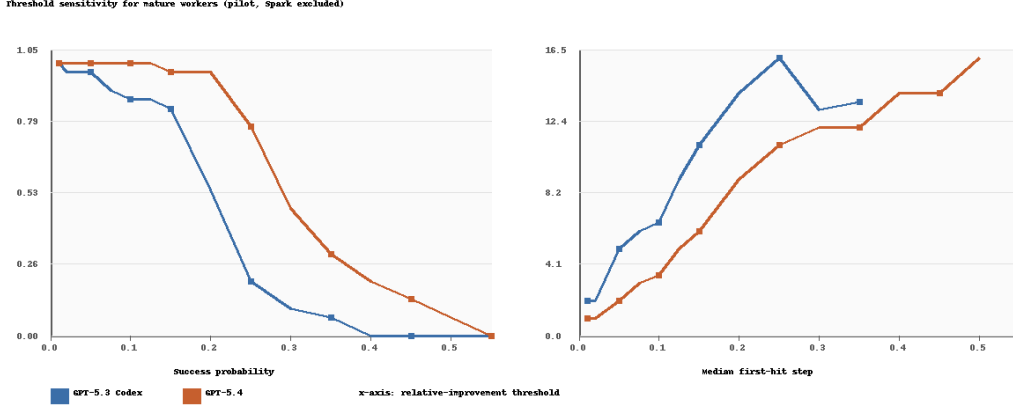


Figure 3: Threshold sensitivity for the same mature-worker comparison. The left panel shows first-passage success probability as the required relative-improvement threshold varies; the right panel shows the median first-hit step. At the paper’s operating point $\delta = 0.05$, gpt_5_4 has higher success probability and lower median first-hit time than gpt_5_3_codex. The fact that this speed advantage does not automatically imply lower deployment loss is the operational distinction the decomposition is meant to expose.

Table 2: Router-facing mode-diagnosis audit on the current signal records. Accuracy is reported only as a diagnostic for whether the signal exposes the finite task-mode structure; it is not the deployment objective.

Signal record	Records	Accuracy	Macro accuracy
Budget only	63	0.349	0.333
Probe only	63	1.000	1.000
Probe + budget	63	1.000	1.000
Leaky current-state control	63	1.000	1.000

321 currently charged by the protocol, the recommendation changes. The experiment therefore already
 322 exhibits the failure mode of scalar leaderboard racing: different components of time-to-certified-
 323 solution point to different engineering choices.

324 **Experiment 2: router-response audit.** For each task mode and each signal Z_j , query the same
 325 orchestrator under the same worker menu. The router returns a structured decision record containing
 326 selected worker, confidence, short task diagnosis, evidence used, expected failure mode, and expected
 327 cost risk. The diagnostic question is whether progressive signals change the router’s own decision
 328 record: selected-worker distribution, confidence, diagnosis consistency with the declared task mode,
 329 and cited evidence. This is a router-response diagnostic, not an estimator of $I_\pi(S; Z)$. An information
 330 estimator would require a predeclared finite representation or predictive model for the signal records.

331 **Current signal-audit results.** The current router-facing diagnostic is not yet a deployment claim.
 332 It asks the narrower question required by the information term: whether admissible pre-deployment
 333 records distinguish the task modes that have different empirical frontiers. On the current snapshot,
 334 budget-only records collapse the three modes to the same predicted class, while probe-containing
 335 records perfectly recover the predeclared mode labels.

336 This supports the premise that low-cost probes can reveal mode information, but it does not yet
 337 prove that the router uses that information correctly. The deployment-facing question is deferred to
 338 Experiments 3 and 4: whether the induced worker choice moves toward the empirical frontier and
 339 pays for the measurement cost.

340 **Experiment 3: routing shift and allocation regret.** Using the decisions from Experiment 2,
 341 compute the empirical panel shift rate

$$\widehat{\text{Shift}}_R(j) = \sum_{s \in \mathcal{S}} \hat{\pi}(s) \mathbf{1}\{\rho_j(\chi_s) \neq \rho_0(\chi_s)\}$$

342 for the deterministic primary router. Then use the frontier from Experiment 1 to score each selected
 343 action by allocation regret

$$\hat{r}_j(\chi_s) = [\log \hat{\kappa}_{\chi_s}(\rho_j(\chi_s)) - \log \tilde{p}_{\chi_s}(\rho_j(\chi_s))] - \min_{a \in \mathcal{A}} [\log \hat{\kappa}_{\chi_s}(a) - \log \tilde{p}_{\chi_s}(a)].$$

344 This is an operational action-frontier diagnostic, not an estimator of the KL allocation-summary
 345 mismatch term $\mathbb{E}_Z \text{KL}(\pi_Z || q_Z)$ unless a separate construction of q_Z has been predeclared. When
 346 the same pilot runs define the frontier and score regret, regret is reported as an in-sample diagnostic;
 347 cross-fitted regret is used whenever enough pilot trajectories are available. Shift rate measures
 348 whether Z_j changes the routing policy, while allocation regret asks whether the changed policy moves
 349 toward the empirical mode-conditional frontier from Experiment 1. A signal can induce a router
 350 response and still increase allocation regret if it makes the orchestrator change worker in the wrong
 351 direction. Conversely, low regret indicates that the selected action moved closer to the empirical
 352 mode-conditional frontier rather than merely increasing decision variability.

353 **Current routing-regret status.** The snapshot has enough signal records to show that probes expose
 354 the predeclared mode structure, but the allocation-regret claim remains conditional on the frozen
 355 worker frontier and the final prompt-only router decisions under the Spark-free worker menu. For this
 356 draft we therefore report the router plots as diagnostics rather than as promoted deployment evidence.
 357 The intended reading is conservative: mode information is available; whether it is useful depends on
 358 whether the router maps that information to the cost-adjusted worker frontier in Figure 2.

359 **Experiment 4: paired selected-worker deployment gain.** Report all $j \in \{1, 2, 3\}$ with multi-
 360 plicity control as the primary analysis. If a single non-static signal is selected on a diagnostic split,
 361 evaluate it only on a separate holdout split. For each task mode and paired deployment index r ,
 362 compare the worker selected under Z_0 with the worker selected under Z_j from the same original
 363 artifact and horizon. Pairing is by common prototype, common checker configuration, and matched
 364 deployment seed when the backend exposes seed control; otherwise intervals are stratified by task
 365 mode rather than treated as seed-paired. The selected-action confirmation uses $Q_{\text{dep}} = 10$ repeats
 366 per selected action and prototype unless a larger holdout budget is declared. Estimate

$$\hat{\Delta}_R(j) = \hat{\ell}_{\text{dep}}(\rho_0) - \hat{\ell}_{\text{dep}}(\rho_j) - \hat{\ell}_{\text{meas}}(j).$$

367 Report confidence intervals over the declared pairing unit, first-pass success changes, cost-to-success
 368 changes, gross loss reduction, measurement cost, and the subset where the signal changed the worker
 369 but increased loss. This experiment is the deployment validation of the theory-facing diagnostics.
 370 Proposition 1 gives the estimand and estimator for the same-orchestrator comparison, while (5) defines
 371 the loss used to decide whether a signal-conditioned policy is actually better after measurement cost is
 372 charged. Positive routing information and lower allocation regret are not sufficient by themselves; the
 373 routed-policy claim is supported only when the paired gain $\hat{\Delta}_R(j)$ is positive on held-out trajectories.

374 **Current validation status.** The non-Spark confirmation block is nearly complete in the current
 375 campaign snapshot: gpt_5_3_codex has 90/90 materialized holdout summaries and gpt_5_4 has
 376 78/90, with the remaining missing summaries concentrated in resnet_micro. The gpt_5_4_mini
 377 validation block is left blank until its pilot cell reaches the same support and the final lower-cost
 378 comparison is frozen. We therefore do not yet claim a positive paired router gain. The current result
 379 is a stronger calibration claim: the mature-worker pilot already produces rank disagreements that the
 380 holdout split is designed to validate or falsify.

381 **Experiment 5: negative controls.** Repeat Experiments 2–4 with schema-preserving controls:
 382 shuffled probe records, probes from the wrong task mode, and synthetic noise records with the
 383 same field names and token length. The same router, prompt, worker menu, and scoring rule are
 384 used. These controls test whether router response, allocation regret, and paired deployment gain
 385 are specific to task-relevant signal content. They preserve the format, approximate context burden,

and measurement/context cost of the corresponding real Z_j while breaking its relationship to the task mode and worker frontier. A credible deployment claim requires the real signal to reduce allocation regret and improve paired deployment gain more than these controls after measurement cost is charged.

Current negative-control status. Negative controls are not yet promoted to the main result. They should remain a claim gate: if shuffled or wrong-mode probe records produce the same router shifts as true probe records, then the router is responding to format or context length rather than task-relevant signal content. In that case the paper should retain the worker-frontier result but weaken the routing-information claim.

Experiment 6: robustness to task-instance variation. The main protocol deliberately fixes `train.py` initialization and data split to isolate agentic trajectory variance. As a robustness appendix, repeat the worker-frontier and paired-routing comparisons with a small grid of additional initialization/data seeds, e.g. three seeds and five trajectories per task-mode-worker cell. These runs are not pooled with the main estimates. They stress-test Assumption 1, which requires the finite mode abstraction to have stable worker success probabilities and costs within each task mode. The additional seeds define nearby executable task instances, not new pooled evidence for the main estimates. If competence frontiers, allocation regret, or paired gains change qualitatively under these modest perturbations, the experiment has found a useful warning: the packed-mode abstraction is too coarse for the claimed theory-facing explanation. Accordingly, the main protocol is a fixed-prototype repeated-run study unless these additional instances are promoted into the main sampling design.

Current robustness status. The main tables in this draft are fixed-mode, fixed-template estimates. Additional seed and split perturbations are reserved for the appendix and should be interpreted as stress tests of Assumption 1, not as extra pooled evidence for the main worker ranking.

Claim promotion gates. We promote a routing recommendation only when the following are reported: mode-conditional competence with uncertainty, worker-side cost sensitivity, router shift, allocation regret, paired deployment gain with measurement cost charged, negative controls, frontier stability, calibration of $\tilde{p}$, and surrogate-versus-loss ranking agreement. If routing diagnostics improve but paired deployment gain does not, the conclusion is diagnostic rather than prescriptive. If paired deployment gain improves but surrogate-versus-loss agreement is poor, the protocol found a useful router but did not validate the packed-mode explanation.

7 Related work

The closest classical ancestor is algorithm selection. Since Rice’s formulation, solver portfolios have treated performance as instance-dependent: SATzilla and AutoFolio learn to select solvers from instance features rather than report one globally best solver [3–6]. That literature also includes feature-computation costs, pre-solving, and probing. Our contribution is the specialization to externally checked LLM agents, where a router observes a controlled pre-deployment record, selects a worker or harness action, and is scored by paired deployment loss rather than by standalone routing accuracy.

LLM routing and cascades provide the model-menu context. FrugalGPT frames model cascades as cost reduction across heterogeneous APIs, RouteLLM learns cost-quality routing from preference data, and recent compound-system routing work extends the idea to multi-module systems [7–11]. These methods establish that heterogeneous model menus can create economic gains. The present paper asks how much checked, task-specific information is worth before committing to an expensive agentic deployment.

Checked agent benchmarks provide the task substrate. SWE-bench evaluates repository repair against tests, AutoResearch evaluates training-loop edits against validation feedback, and theorem-proving systems use analogous external validators [1, 12]. Most benchmarks ask how well a worker performs after it is launched. Our question is upstream: which worker should be launched, what should the router be allowed to observe, and how should the cost of that observation be charged?

Finally, repeated sampling and inference-time scaling show that lower-cost or smaller models can become competitive when verification is available [13–16]. The first-passage retry crossover in

436 Theorem 2 is the simplest version of that tradeoff. The four-term identity embeds the same idea in a
437 mode-aware routing and measurement account.

438 8 Discussion, future directions, and conclusion

439 The paper has two levels of claim. The theory-facing claim is an accounting identity under Assump-
440 tion 1. It is exact, but only for the certified log-effort surrogate. The deployment-facing claim is
441 empirical: the accounting should predict or explain useful action selection under a richer deployment
442 loss. These two claims should not be collapsed. The current experiments already support the weaker
443 but central diagnostic claim: raw worker rankings, first-passage rankings, log-effort rankings, and
444 deployment-loss rankings can disagree on the same checked task family. That disagreement is
445 precisely the regime in which a budget-aware orchestration framework is needed.

446 The packed-mode assumption is the main modeling limitation. Real agent trajectories are stateful,
447 attempts are not independent, and task modes may be fuzzy. The paper therefore reports first-passage
448 curves, occupancy, hazard diagnostics, frontier stability, cost sensitivity, and surrogate-versus-loss
449 ranking agreement instead of assuming that the finite-mode surrogate is correct. If these diagnostics
450 are unstable, the decomposition remains a useful diagnostic language but not a reliable selection rule.

451 The operational task-mode schema is also a limitation. If S is too coarse, mode-conditional compe-
452 tence is washed out and the router appears useless. If S is too fine, sample sizes per cell collapse and
453 the signal-mode estimates become unstable. A full evaluation therefore includes an ablation over
454 task-mode schemas, including task-family-only, probe-bin-only, and crossed schemas.

455 Finally, measurement value is router-specific. A baseline probe can contain information about the
456 best action and still fail to improve deployment loss if the actual router does not use it well or if
457 the measurement is too expensive. For this reason, the paired gain $\Delta_R(j)$ is the main deployment
458 estimand, while mutual-information diagnostics, allocation-summary mismatch, and allocation regret
459 help explain why that gain appears or fails to appear.

460 **Future directions.** The immediate experimental next step is to finish the gpt_5_4_mini pilot
461 and holdout cells, replacing the unavailable Spark worker with a stable lower-cost action. That will
462 make the worker menu span a genuine cost-capability frontier rather than only two strong models.
463 The second next step is to freeze the Spark-free router prompt and rerun the Z_0 – Z_3 routing audit
464 against the final three-worker menu. Only then should the paired deployment-gain claim be promoted.
465 Beyond this benchmark, the same protocol should be tested on repository repair, theorem checking,
466 and multi-file research-code editing, where the mode schema is less artificial and measurement costs
467 are more heterogeneous.

468 **Conclusion.** Model choice inside an agent harness is not a scalar leaderboard problem. It is a
469 measurable decomposition problem validated by paired deployment outcomes. The right worker
470 depends on task state, checked competence, routing information, routing mismatch, first-passage
471 time, retry depth, and worker-side cost. Our CIFAR-10 edit-verify campaign provides the first
472 empirical slice of that argument: the same two mature workers can trade off speed, final improvement,
473 log-effort, and deployment loss in different ways. The final claim will be strongest if the lower-cost
474 gpt_5_4_mini cells and the paired router validation preserve this rank-disagreement pattern while
475 showing that pre-deployment information can move allocation toward the cost-adjusted frontier.

476 References

- 477 [1] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and
478 Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In
479 *International Conference on Learning Representations (ICLR)*, 2024.
- 480 [2] Alessandro Achille and Stefano Soatto. AI agents as universal task solvers: It’s all about time.
481 *Entropy*, 28(3):332, 2026.
- 482 [3] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages
483 65–118. Elsevier, 1976.

- [4] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- [5] Marius Lindauer, Katharina Eggersperger, Matthias Feurer, Bernd Bischl, and Frank Hutter. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53:745–778, 2015.
- [6] Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(3):48–60, 2016.
- [7] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [8] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.
- [9] Lingjiao Chen et al. Optimizing model selection for compound AI systems. *arXiv preprint arXiv:2502.14815*, 2025.
- [10] Yanwei Yue, Guibin Zhang, Boyang Liu, Guancheng Wan, Kun Wang, Dawei Cheng, and Yiyan Qi. MasRouter: Learning to route LLMs for multi-agent systems. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15549–15572, 2025.
- [11] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Chris Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research Blog, 2024.
- [12] Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, 2026. GitHub repository.
- [13] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- [14] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations (ICLR)*, 2025.
- [15] Yangzhen Wu, Zhiqing Sun, Shanda Li, Sean Welleck, and Yiming Yang. Inference scaling laws: An empirical analysis of compute-optimal inference for problem-solving with language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [16] Jinghan Zhang, Kunhao Zheng, Yici Yan, Yuchen Ouyang, and Jun Zhu. Scaling LLM inference with optimized sample compute allocation. *arXiv preprint arXiv:2410.22480*, 2024.

518 A Proof details

519 A.1 Four-term identity

520 Under Assumption 1,

$$\log T_{\rho,q}(S, Z) = C + \log \kappa(a_Z, S) - \log p(a_Z, S) - \log q_Z(S).$$

521 Conditioning on $Z = z$,

$$\mathbb{E}[-\log q_z(S) \mid Z = z] = \sum_s \pi_z(s) \log \frac{1}{q_z(s)} = H(\pi_z) + \text{KL}(\pi_z \parallel q_z).$$

522 Therefore

$$\mathbb{E}[\log T_{\rho,q}] = C + \mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} [\log \kappa(a_Z, S) - \log p(a_Z, S)] + \mathbb{E}_Z [H(\pi_Z)] + \mathbb{E}_Z [\text{KL}(\pi_Z \parallel q_Z)].$$

523 For the prior-matched baseline ρ_0 that deploys fixed action a_0 and uses $q_0(\cdot | z) \equiv \pi$,

$$\mathbb{E}[\log T_{\rho_0, \pi}] = C + \mathbb{E}_{S \sim \pi}[\log \kappa(a_0, S) - \log p(a_0, S)] + H(\pi).$$

524 Subtracting the deployed effort from the baseline effort yields

$$\Delta = \mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} \left[\log \frac{\kappa(a_0, S)}{\kappa(a_Z, S)} \right] + \mathbb{E}_Z \mathbb{E}_{S \sim \pi_Z} \left[\log \frac{p(a_Z, S)}{p(a_0, S)} \right] + H(\pi) - \mathbb{E}_Z[H(\pi_Z)] - \mathbb{E}_Z[\text{KL}(\pi_Z \| q_Z)].$$

525 The entropy difference is $I_\pi(S; Z)$, giving (4).

526 A.2 Pilot concentration

527 For binary success observations in an action-mode cell, Hoeffding gives

$$\mathbb{P}(|\hat{p}(a, s) - p(a, s)| > \alpha) \leq 2 \exp(-2n_0\alpha^2).$$

528 If all true probabilities are at least $p_{\min}$, then the log-probability error obeys $|\log \hat{p} - \log p| \leq$
 529 $\alpha/p_{\min} + O(\alpha^2/p_{\min}^2)$ on the good event. A union bound over $|\mathcal{A}| |\mathcal{S}|$ cells yields the conservative
 530 pilot rule

$$n_0 = O\left(\frac{\log(|\mathcal{A}| |\mathcal{S}| / \delta_{\text{conf}})}{\delta_{\text{acc}}^2 p_{\min}^2}\right).$$

531 Wilson intervals and bootstrap intervals are reported in practice because pilot cell counts are small.

532 B Operational diagnostics for the four terms

533 The identity above becomes useful only after each term is tied to quantities logged by the deployment
 534 protocol. The checker-facing progress process is the common object used to operationalize compe-
 535 tence, cost, information, and allocation diagnostics. Fix a horizon H and a task-normalized checked
 536 progress process $G_t(a, x) \in [0, 1]$ for action a on instance x . In the main experiment, the router acts
 537 once before deployment: after observing Z , it selects one action, and that action is used for the whole
 538 trajectory. The success and cost diagnostics below are therefore trajectory-level quantities computed
 539 from the step-wise verifier trace. For lower-is-better checker losses, one admissible normalization is

$$G_t(a, x) = \text{clip}_{[0,1]} \left(\frac{L_0(x) - L_t(a, x)}{|L_0(x)| + \varepsilon_L} \right),$$

540 where $L_0(x)$ is the starting-artifact loss and $L_t(a, x)$ is the checked loss convention at step t . Given a
 541 success threshold $\delta > 0$,

$$\tau_\delta(a, x) = \inf\{t \leq H : G_t(a, x) \geq \delta\}, \quad F_\delta(a, x) = \mathbf{1}\{\tau_\delta(a, x) = \infty\}.$$

542 We also log persistence through

$$\text{Occ}_{\delta, H}(a, x) = \frac{1}{H} \sum_{t=1}^H \mathbf{1}\{G_t(a, x) \geq \delta\}.$$

543 **Competence.** The action competence component $p(a, s)$ is the first-success probability of action a
 544 on task mode s :

$$p(a, s) = \mathbb{P}[\tau_\delta(a, X) \leq H \mid S = s].$$

545 Thus G_t is step-wise, but the policy competence term uses the probability that the worker selected by
 546 the router reaches the success threshold at least once before the horizon. Cumulative progress and
 547 persistence are not substituted for $p(a, s)$ in the identity. We log

$$\bar{G}_a(s) = \mathbb{E} \left[\frac{1}{H} \sum_{t=1}^H G_t(a, X) \mid S = s \right]$$

548 and $\text{Occ}_{\delta, H}$ as secondary diagnostics, especially when first-hit success saturates or when partial
 549 progress is scientifically relevant.

550 **Cost.** The action cost component $\kappa(a, s)$ is kept separate from competence. In the main protocol
 551 it is the normalized cost of deploying action a on mode s for the same trajectory convention used
 552 to define $p(a, s)$. The routed-policy cost term averages this mode-conditional cost over the actions
 553 selected under Z . The primary convention is cost accumulated until the first certified success or the
 554 horizon, $\tau_\delta \wedge H$. We log

$$C_\delta(a, x; \xi) = \lambda^\top (\tilde{T}_{\tau_\delta \wedge H}^{\text{worker}}, \tilde{C}_{\tau_\delta \wedge H}^{\text{tok}})(a, x; \xi),$$

555 and report worker wall-clock cost as primary, with token count as a sensitivity analysis. Checker
 556 runtime is logged for audit, but it is not part of $\kappa(a, s)$ unless the checker protocol or checker-call
 557 frequency differs across actions.

558 **Routing information.** The theorem term $I(S; Z)$ measures how much a router-visible signal Z
 559 separates the predeclared task modes before deployment. This does not mean that the evaluator
 560 already knows the best worker for each instance. The evaluator knows only the candidate mode
 561 schema and can estimate whether Z_j makes that schema more identifiable:

$$I(S; Z_j | Z_0) = H(S | Z_0) - H(S | Z_j).$$

562 The quantity is estimated only when a finite signal representation or calibrated predictive model for
 563 the signal records is predeclared. Otherwise the paper reports signal summaries and router-response
 564 diagnostics instead of a Shannon mutual-information estimate. It is a descriptive information
 565 diagnostic; its decision value is established only after action outcomes show that the modes predict
 566 different cost–success profiles. The paired router experiment then checks whether higher-information
 567 signals actually change the selected action and improve first-passage deployment outcomes.

568 **Allocation-summary mismatch and allocation regret.** The theorem term $\mathbb{E}_Z \text{KL}(\pi_Z \| q_Z)$ is
 569 defined only after the evaluator predeclares a reproducible construction of the allocation summary
 570 q_Z . Operationally, the router returns an action, not a mode label or distribution. When no such q_Z
 571 construction is frozen, the experiment reports allocation regret as a separate action-frontier diagnostic.
 572 After deployment runs, the evaluator estimates the mode-conditional frontier

$$a^*(s) \in \arg \min_{a \in \mathcal{A}} \{\log \kappa(a, s) - \log p(a, s)\},$$

573 or the statistically indistinguishable set of frontier actions. For a selected action $a = \rho_j(x)$ and mode
 574 $s = \sigma(x)$, the corresponding allocation regret diagnostic is

$$r_j(x) = [\log \kappa(a, s) - \log p(a, s)] - \min_{a' \in \mathcal{A}} [\log \kappa(a', s) - \log p(a', s)].$$

575 This regret is computed retrospectively on a diagnostic split and validated on holdout instances; it
 576 is not an oracle input to the router. High information with high allocation regret means that the
 577 progressive signal exposed useful mode structure, but the induced action did not exploit the empirical
 578 cost–success frontier.

579 After these diagnostics are estimated, the experiment still reports the decision-facing first-passage
 580 deployment loss in (5) as final validation. The full-horizon audit loss in (6) is reported separately for
 581 persistence and final-quality checks. These losses are not replacements for the theorem; they test
 582 whether designs favored or explained by the diagnostics improve the practical deployment objective.

583 C Operational decomposition under progressive signals

584 The main four-term identity compares a deployed design to a prior-matched baseline. For the V3
 585 router experiment, it is often useful to compare two progressive signal conditions for the same router.
 586 Assume a finite task mode $S \in \mathcal{S}$ and, for each signal condition j , the conditional mode distribution
 587 induced by the realized signal record

$$\pi_j(s | Z_j) = \mathbb{P}[S = s | Z_j].$$

588 The fixed orchestrator R induces the routing policy $a_R^j(Z_j) = R(Z_j)$. If a mode-facing allocation
 589 summary $q_R^j(\cdot | Z_j) \in \Delta(\mathcal{S})$ has been predeclared, define

$$\mathcal{E}_R^j(s, Z_j) = \frac{\kappa(a_R^j(Z_j), s)}{p(a_R^j(Z_j), s) q_R^j(s | Z_j)}.$$

Table 3: Frozen operational configuration for the main protocol. All values are fixed within a task mode; repeated runs vary only the agentic trajectory.

Component	Fixed value
Dataset and split	CIFAR-10, 50,000 training examples and 10,000 validation examples; balanced subset, label noise 0.0, imbalance ratio 1.0; the train/validation split is fixed in the main protocol
Data seed	Common fixed data/subset seed; alternative split seeds are reported only as robustness variants
<code>train.py</code> initialization seed	SEED=42; used for Python, NumPy, PyTorch, CUDA seeds, deterministic PyTorch mode, and the training-loader generator; alternative initialization seeds are robustness variants
Training transforms	random crop with padding 4, random horizontal flip, CIFAR-10 normalization; validation uses normalization only
Training budget per checker call	fixed-step mode with <code>A_MAX_STEPS</code> =256; time budget 120s is a fallback, not the primary scoring convention
Common hyperparameters	<code>BATCH_SIZE</code> =64, <code>NUM_WORKERS</code> =0, Adam, learning rate $5 \cdot 10^{-4}$, weight decay 10^{-4} , betas (0.9, 0.999), no LR schedule
Task mode	<code>mlp_flat</code> , <code>cnn_compact</code> , or <code>resnet_micro</code>
Shared template parameters	depth 2, base channels 12, channel multiplier 2, batch norm enabled, dropout 0.0, and hidden width 48
Agent horizon	$H = 20$ edit-verify steps; trajectories continue to H after first success to measure persistence and final quality
Progress normalization	$\varepsilon_L = 10^{-8}$ in the denominator of G_t
Primary loss weights	$\alpha_{\text{fail}} = 1$; normalized worker wall-clock is the primary scalar C_δ ; token cost is a sensitivity analysis
Audit loss weights	$\alpha_{\text{occ}} = \alpha_{\text{qual}} = 1$, $\psi(u) = 1 - u$
Measurement cost	pre-deployment probes are charged through ℓ_{meas} using the same wall-clock normalizer as C_δ ; context-token cost is reported as sensitivity
Router configuration	deterministic prompt-only router with fixed model identifier, prompt hash, parser, temperature, top-p, and seed policy recorded before evaluation

590 The log-effort gain of signal condition Z_j over Z_0 is

$$\Delta_R^{\log}(j) := \mathbb{E}[\log \mathcal{E}_R^0(S, Z_0) - \log \mathcal{E}_R^j(S, Z_j)].$$

591 Expanding the cross-entropy terms gives

$$\Delta_R^{\log}(j) = C_R(j) + \Phi_R(j) + G(j) + M_R(j),$$

592 where

$$\begin{aligned} C_R(j) &= \mathbb{E} \left[\log \frac{\kappa(a_R^0(Z_0), S)}{\kappa(a_R^j(Z_j), S)} \right], \\ \Phi_R(j) &= \mathbb{E} \left[\log \frac{p(a_R^j(Z_j), S)}{p(a_R^0(Z_0), S)} \right], \\ G(j) &= H(S \mid Z_0) - H(S \mid Z_j), \\ M_R(j) &= \mathbb{E} \left[\text{KL}(\pi_0(\cdot \mid Z_0) \parallel q_R^0(\cdot \mid Z_0)) - \text{KL}(\pi_j(\cdot \mid Z_j) \parallel q_R^j(\cdot \mid Z_j)) \right]. \end{aligned}$$

593 When Z_j refines Z_0 , $G(j) = I(S; Z_j \mid Z_0)$. This signal-mode decomposition is theory-facing and
594 excludes measurement cost unless such a term is added explicitly. It is a diagnostic companion to the
595 deployment-gain estimand (8), not a replacement for it.

596 D Operational estimator details

597 **Trajectory record.** Each run stores the worker alias, resolved model identifier, API date, split,
598 task family, task mode, seed, maximum horizon H , signal condition, prompt hash, router model

599 identifier, router temperature/top-p, router seed policy, parser version, router output, selected worker,
600 and cost convention. Each step stores the prompt-visible state, proposed `train.py` artifact, parse
601 status, checker result, validation loss, validation accuracy, relative improvement, incremental worker
602 wall time, token usage, and checker runtime for audit. Parser failures are logged as unsuccessful
603 routing or deployment records under the predeclared parser-failure policy.

604 **Success and time-to-verification.** For trajectory i , τ_i is the first step at which the relative improve-
605 ment threshold is crossed; censored failures have $\tau_i = \infty$. For each action-mode cell,

$$\hat{F}_a(s, h) = \frac{1}{n_{a,s}} \sum_i \mathbf{1}\{\tau_i \leq h\}, \quad \hat{S}_a(s, h) = 1 - \hat{F}_a(s, h).$$

606 The empirical hazard is

$$\hat{\lambda}_a(s, h) = \frac{\sum_i \mathbf{1}\{\tau_i = h\}}{\sum_i \mathbf{1}\{\tau_i \geq h\}}.$$

607 The raw first-success estimate is $\hat{p}(a, s) = \hat{F}_a(s, H)$. For log-effort scores, the protocol uses the
608 predeclared smoothed estimate

$$\tilde{p}(a, s) = \frac{k_{a,s} + 1/2}{n_{a,s} + 1},$$

609 where $k_{a,s}$ is the number of first successes in the cell. This prevents zero-success pilot cells from
610 making log scores infinite; raw success counts are still reported.

611 **Cost.** The primary cost is cumulative worker wall-clock time to $\tau \wedge H$. Secondary worker-side
612 cost is cumulative token count. Checker runtime is logged as a protocol audit field and excluded from
613 action-cost comparisons unless the checker protocol differs across actions. All plots that support
614 model recommendations must be reproducible under at least worker wall-clock and token-count
615 conventions. When a worker fails before producing a parseable artifact, its cost is still counted and
616 the step is marked unsuccessful. The scalar estimator matched to Assumption 1 is the cell mean

$$\hat{\kappa}(a, s) = \frac{1}{n_{a,s}} \sum_{i: S_i = s} C_{\delta,i}(a),$$

617 or its fixed-prototype analogue $\hat{\kappa}_{\chi_s}(a)$ in Section 6.

618 **Information, allocation summaries, and regret.** For each progressive signal Z_j , a Shannon
619 information estimate is reported only if the protocol predeclares a finite signal representation or
620 calibrated predictive model for estimating $\pi_j(\cdot | Z_j)$ on held-out instances. Otherwise the report
621 uses qualitative signal summaries and router-response diagnostics without calling them estimates
622 of $I(S; Z_j)$. The KL allocation-summary mismatch term is reported only if the evaluator also
623 freezes a reproducible construction of $\hat{q}_R^j(\cdot | Z_{j,i})$ before seeing evaluation outcomes. In the main
624 fixed-prototype protocol, the primary operational diagnostic is allocation regret,

$$\hat{r}_j(x_i) = [\log \hat{\kappa}(a_{j,i}, s_i) - \log \tilde{p}(a_{j,i}, s_i)] - \min_{a \in \mathcal{A}} [\log \hat{\kappa}(a, s_i) - \log \tilde{p}(a, s_i)],$$

625 where $a_{j,i} = R(Z_j(x_i))$ and $s_i = \sigma(x_i)$.

626 **Uncertainty.** Success probabilities use Wilson intervals. Aggregated objectives, paired gains, and
627 calibration diagnostics use bootstrap or randomization tests at the sampling unit declared for the study.
628 If the main protocol remains a fixed-prototype repeated-run panel, paired randomization tests over
629 repeated trajectories are descriptive for that panel and should not be interpreted as population-level
630 inference. If independent task instances are added, uncertainty is reported at the task-instance level.

631 E Additional diagnostic plots

632 This appendix collects diagnostics generated from the current campaign snapshot. Some plots still
633 include partial or legacy cells from the campaign logs; the main text uses the Spark-free mature-worker
634 figures in Figures 2 and 3 for the promoted comparison.

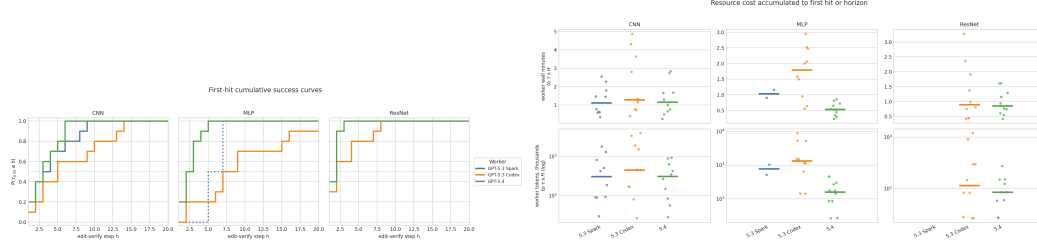


Figure 4: First-passage diagnostics. Left: first-hit CDFs by mode. Right: cost-to-threshold by mode and worker. These are the direct empirical objects behind the retry and time-to-certified-solution interpretation.

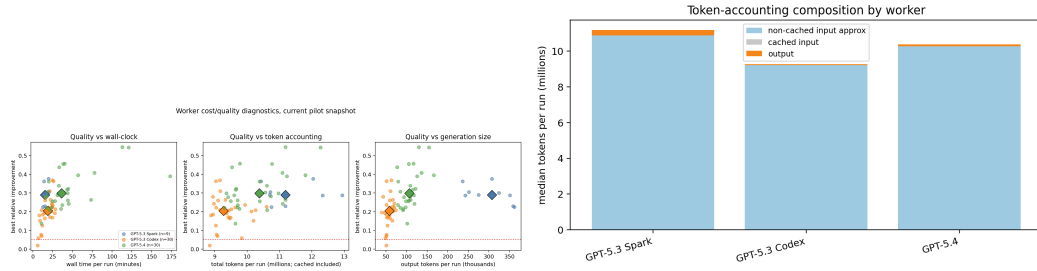


Figure 5: Worker-side cost and accounting diagnostics. These plots check whether conclusions are driven by wall-clock cost, token composition, or quality differences.

F Checklist notes for finalization

Before submission, the checklist should be updated to reflect the final state of the experiments. The current draft contains the theory, operational protocol, statistical plan, and falsification criteria needed for a full NeurIPS-style writeup, but claims should cite exact frozen results only after the final campaign is complete.

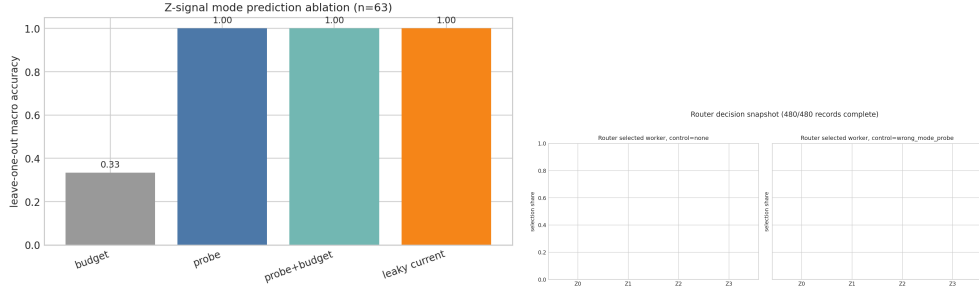


Figure 6: Router-facing diagnostics. The signal ablation tests whether progressively richer records expose task-mode structure; the router-selection snapshot checks whether the prompt-only router changes action distribution under those records.

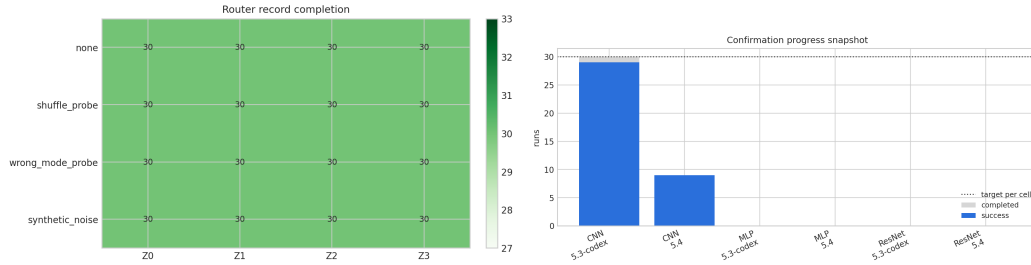


Figure 7: Campaign-completion diagnostics for router records and holdout confirmation runs. These plots are included to make clear which claims are mature and which remain gated by missing cells.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the paper’s scope: verifiable agentic systems, a packed latent-mode assumption, decomposition-guided selection, and a controlled fixed-prototype experimental protocol. The discussion section separates theory-facing and deployment-facing claims.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 8 discusses the packed-mode assumption, small live-agent sample sizes, finite design menus, oracle mode labels, and current cost-measurement limitations.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: Assumption 1 states the assumptions for the main theoretical result, Theorem 1 includes a proof sketch, and Appendix A gives the algebraic proof and pilot concentration details.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 6 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, horizons, metrics, and evaluation protocol; Appendix D specifies the required logs, estimators, and run-metadata fields.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The paper is written against an existing repository and includes reproducibility commands, but an anonymized public code/data release is not included in the current submission package.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.

- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: Section 6 describes the AutoResearch task instantiation, task modes, model menu, router-visible signals, primary estimators, horizons, and evaluation design. Additional implementation details are listed in Appendix D.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: Section 6 and Appendix D specify Wilson intervals, bootstrap intervals, survival curves, calibration diagnostics, and held-out evaluation for the reported protocol.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not include experiments.
- The authors should answer [\[Yes\]](#) if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[No\]](#)

Justification: The paper reports wall-clock costs and run counts but does not provide a complete compute-resource table for every experiment.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work studies verifiable benchmark tasks and does not involve human subjects, sensitive personal data, or released high-risk models. The submission preserves anonymity and follows NeurIPS code and data policies.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 8 notes positive uses in cost-aware and transparent agent deployment as well as limitations and failure modes. The work is methodological and does not introduce a new generative model or dataset of people.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a pre-trained model, image generator, scraped dataset, or other asset with high misuse risk. It evaluates agent harnesses on synthetic or generated benchmark tasks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The related-work section cites existing papers and systems used for positioning, and the experimental section describes the benchmark surfaces. Released code and benchmark assets include explicit license details.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The work introduces a code harness and analysis artifacts as research assets, and Appendix D documents the required logging and configuration fields.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or research with human subjects, so IRB or equivalent approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are central to the evaluated agent designs. Section 6 describes the three-worker menu, fixed prompt-only router, sequential signal protocol, model-identifier logging, and routing-output requirements.

Guidelines:

- 980 • The answer [N/A] means that the core method development in this research does not
981 involve LLMs as any important, original, or non-standard components.
- 982 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
983 be described.